

WHY BIG DATA

From business problems
to distributed systems

Master Data Engineering

Session 1 • 1h30 cours + 2h TP



Ce que vous devez comprendre aujourd'hui

Le premier cours pose le raisonnement qui mène vers Hadoop.

- 01 Reconnaître une situation Big Data
- 02 Distinguer Big Data et BI
- 03 Comprendre Scale-Up vs Scale-Out
- 04 Comprendre pourquoi Google a inventé GFS + MapReduce
- 05 Situer HDFS, YARN, MapReduce, Hive et HBase

PART I

Le problème

Imaginez : vous êtes Data Engineer chez NovaBank

Une institution financière moderne produit des données en continu.

2 TB

de journaux techniques
chaque jour

Mobile

API Paiement

Web banking

ATM

Back-office



Question

Où stocker tout cela pendant 5 ans ?

Is this Big Data?

Avant de définir, on questionne les intuitions.

Cabinet médical

5 GB/day

Dossiers PDF + Excel

E-commerce

400 GB/day

Transactions + clics

NovaBank

2 TB/day

Logs + transactions

IoT Fleet

20 GB/day

1 million events/min

À partir de quel cas parle-t-on vraiment de Big Data ?

100 GB ?

100 TB ?

1 TB ?

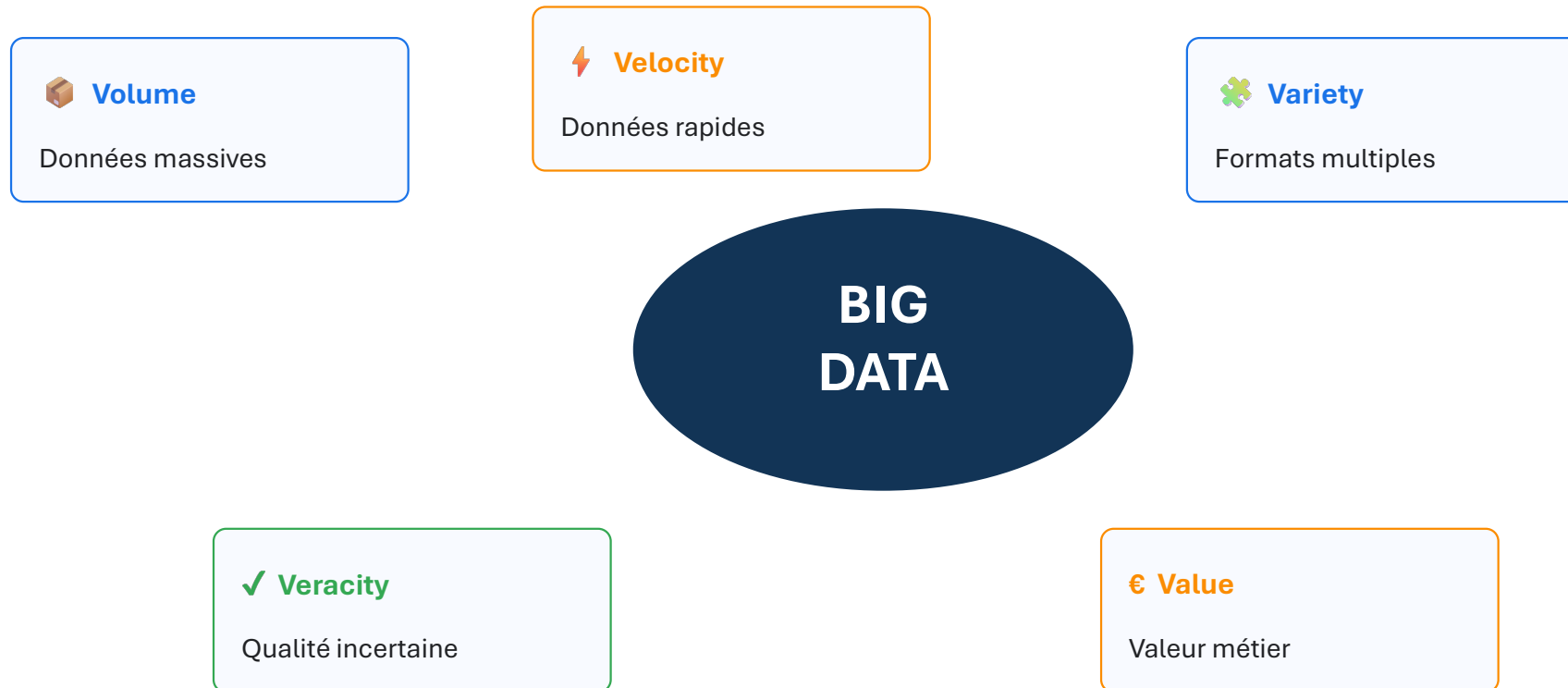
1 PB ?

Il n'existe pas de seuil officiel.

On parle de Big Data quand les technologies classiques ne suffisent plus.

What makes data “Big”?

Les 5V servent à diagnostiquer une situation Big Data.



Le but du Big Data

Transformer des données brutes en valeur métier.



Fraud detection

Détecter les anomalies

Risk analysis

Réduire l'exposition

Automation

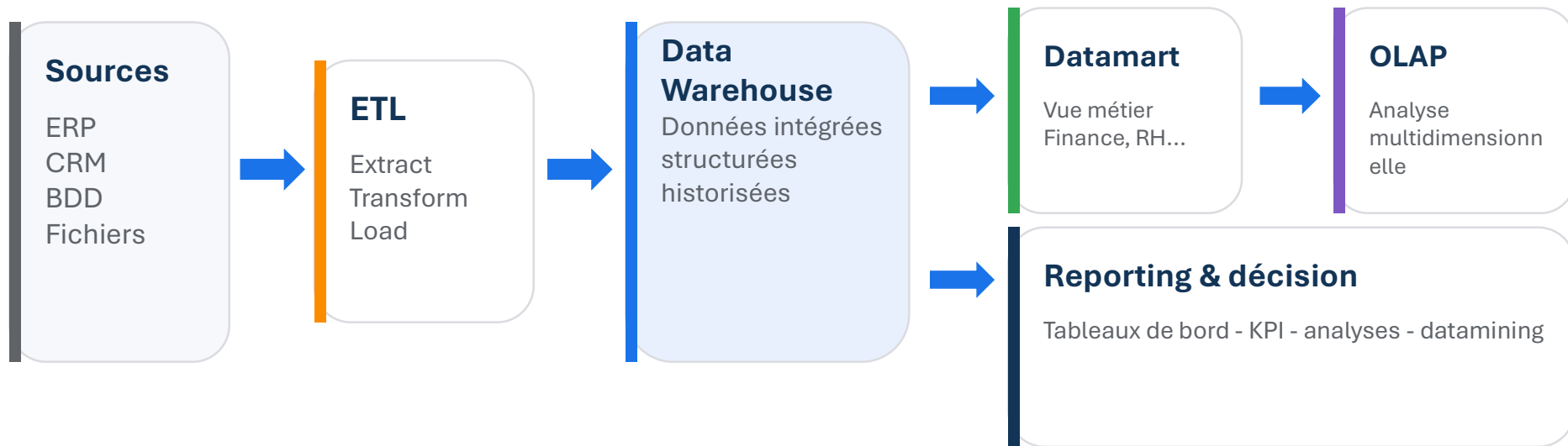
Industrialiser les traitements

AI readiness

Préparer les données IA

Avant le Big Data : la Business Intelligence

Structurer les données de l'entreprise pour analyser et décider.



La BI transforme des données consolidées en indicateurs utiles à la prise de décision.

Big Data ≠ Business Intelligence

Les deux approches sont complémentaires, pas concurrentes.

Business Intelligence

Reporting

Données structurées

Data Warehouse

SQL classique

Tableaux de bord

Big Data

Volumes distribués

Données variées

Data Lake

Traitements distribués

IA / Analytics

Question : Big Data remplace-t-il la BI ?

Business Intelligence et Big Data : le point commun

Des technologies différentes, mais le même objectif : créer de la valeur avec les données.



Récupérer, traiter et comprendre les données pour améliorer la performance et la prise de décision.

Terminologie Big Data : Batch et Batch Processing

Accumuler un lot de données, puis déclencher un traitement global.

Batch

Un lot de données ou de tâches traité comme une unité.

Exemples : paie mensuelle, logs quotidiens.



Batch Processing

Traitement différé et massif. On privilégie souvent le débit global plutôt que la réponse immédiate.

MapReduce est historiquement un moteur de traitement batch distribué.

Terminologie Big Data : Cloud Computing et Data Lake

Deux notions différentes : où l'infrastructure s'exécute et où les données sont centralisées.

Cloud Computing

Consommer à distance des ressources informatiques sous forme de services.

- Calcul / VM
- Stockage
- Réseau
- Bases de données

Exemples : Scaleway, AWS, Azure, GCP

Data Lake

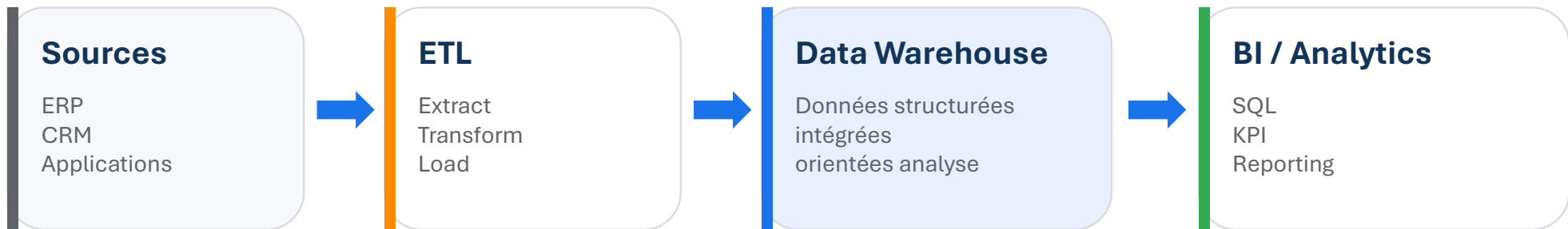
Espace centralisé qui conserve de grands volumes de données dans différents formats, souvent dans leur forme brute ou peu transformée.

CSV - JSON - logs - Parquet - images - IoT

Cloud ≠ Data Lake : le Cloud est un mode de consommation d'infrastructure ; le Data Lake est une architecture de données.

Terminologie Big Data : Data Warehouse, ETL et In-Memory

Trois notions utiles pour comprendre l'évolution vers les plateformes Data modernes.



In-Memory Computing

Conserver ou réutiliser des données intermédiaires en mémoire afin de limiter certaines lectures/écritures disque. Spark exploite fortement cette possibilité, mais peut aussi utiliser le disque.

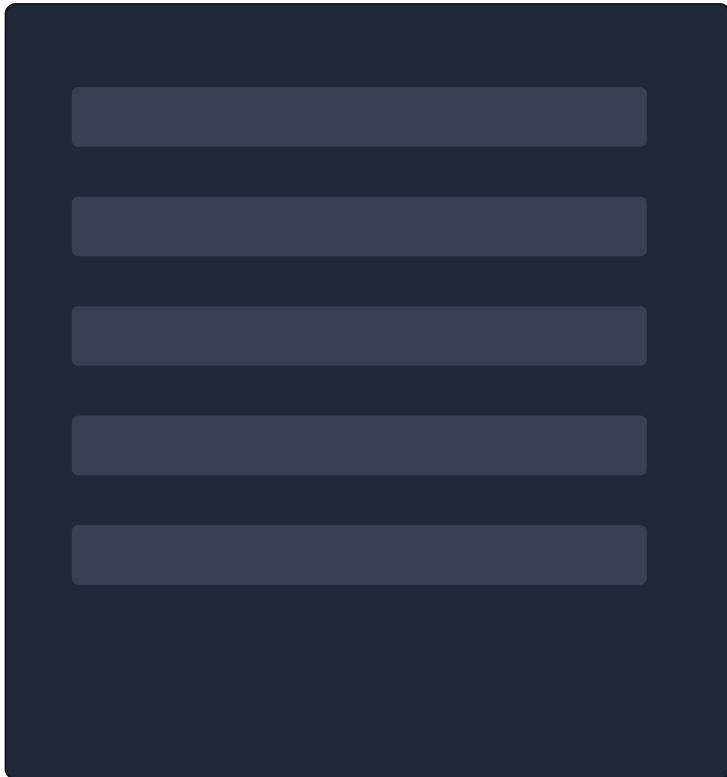
Data Warehouse = données préparées pour l'analyse. ETL = pipeline de préparation. In-Memory = stratégie d'exécution.

PART II

La mauvaise solution

Let's buy a bigger server!

Réflexe classique : faire du Scale-Up.



CPU ↑

RAM ↑

SSD ↑

€ € € € €

Problem solved?

The limits of Scale-Up

Une seule machine devient vite un risque.

✗ Coût

Le prix croît très vite

✗ Limites matérielles

CPU/RAM/disques ne sont pas infinis

✗ Point unique de panne

Une panne bloque tout

✗ Maintenance

Interventions plus risquées

Question : que se passe-t-il si le serveur tombe ?

PART III

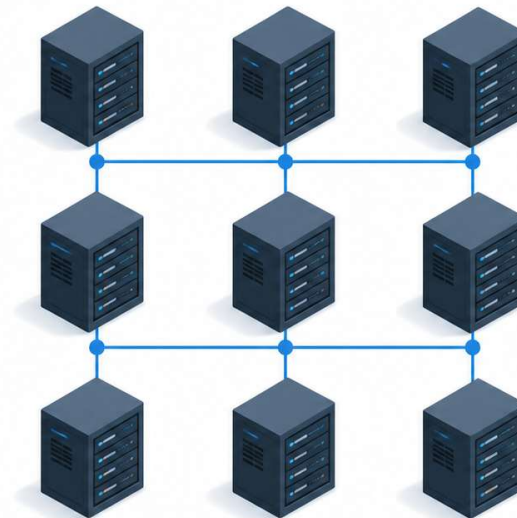
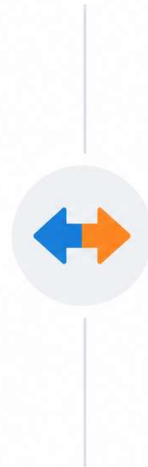
Changer de paradigme

Instead of one big server...

On ajoute des machines standards.



Scale-Up



Scale-Out

Distributed computing changes the rules

Mais un cluster apporte de nouveaux problèmes.

Data locality

Déplacer le calcul vers les données

Fault tolerance

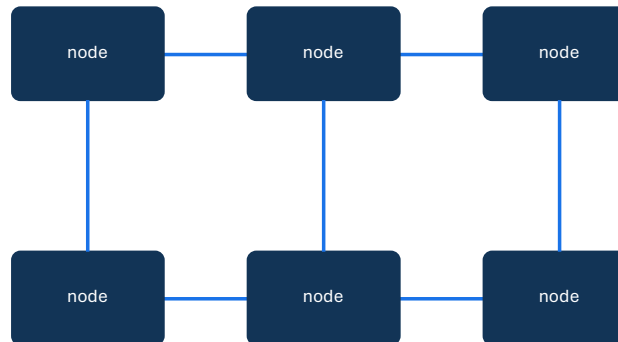
Continuer malgré les pannes

Parallelism

Découper le travail

Coordination

Gérer les ressources partagées



Théorème CAP : un compromis des systèmes distribués

Lors d'une partition réseau, un système distribué doit arbitrer entre cohérence et disponibilité.

C - Consistency

Tous les clients observent une vue cohérente de la donnée après une écriture réussie.

A - Availability

Chaque requête reçoit une réponse, même si certains nœuds ne peuvent plus communiquer.

P - Partition Tolerance

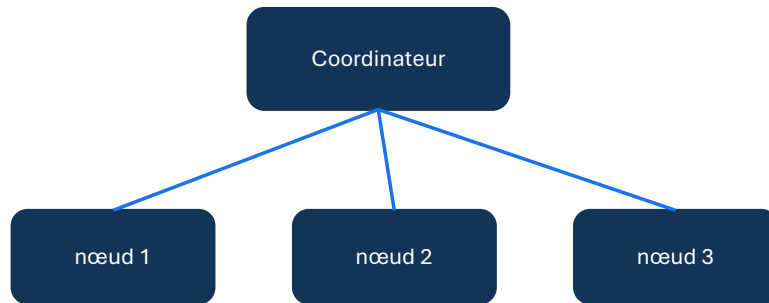
Le système continue à fonctionner malgré une coupure réseau entre des groupes de nœuds.

**CAP ne signifie pas "choisir 2 propriétés sur 3 en permanence".
Le vrai arbitrage apparaît lorsqu'une partition réseau se produit.**

Distribution horizontale : centralisée vs décentralisée

Le Scale-Out distribue les données et le calcul. La coordination peut rester centralisée ou être répartie.

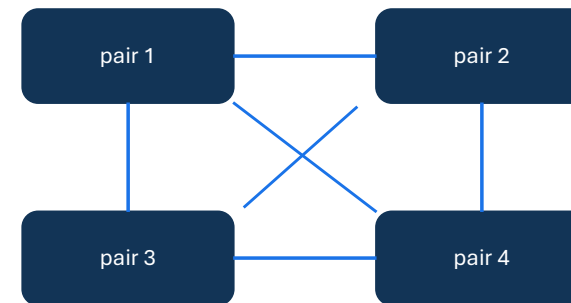
Centralisée



Décision centrale Un coordinateur pilote les décisions globales du cluster.

Impact Simple à administrer, mais le coordinateur doit être rendu hautement disponible.

Décentralisée



Décision répartie Plusieurs nœuds participent à la coordination et au placement.

Impact Plus résilient au contrôle central, mais cohérence et consensus sont plus complexes.

À retenir : un système distribué n'est pas forcément décentralisé.

PART IV

Google invente le modèle

Google's challenge

Indexer le Web avec des machines qui tombent en panne.

billions

de pages à crawler

cheap

machines standards

failures

pannes fréquentes

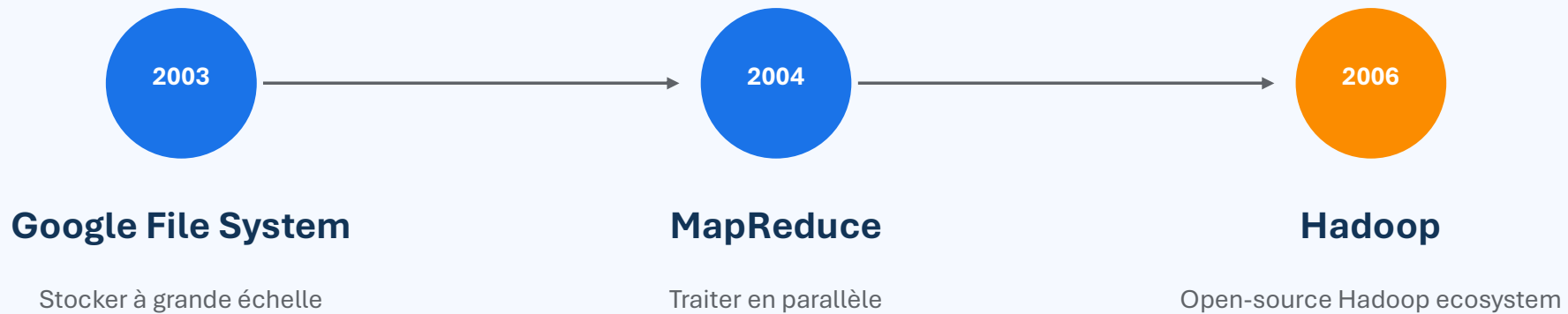
parallel

calcul distribué

Comment construire un moteur de recherche si les machines ne sont jamais totalement fiables ?

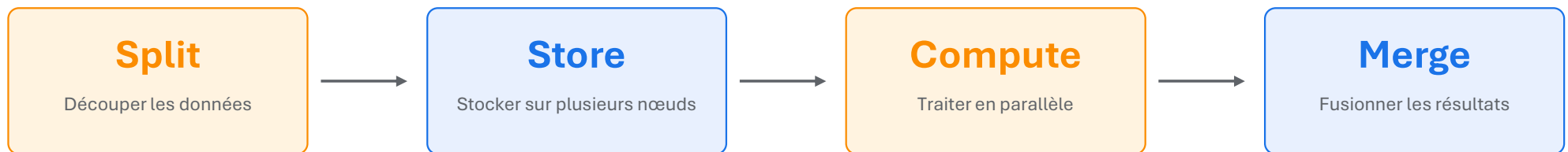
From Google papers to Hadoop

Les idées deviennent un écosystème open-source.



The big idea

Découper, distribuer, traiter localement.



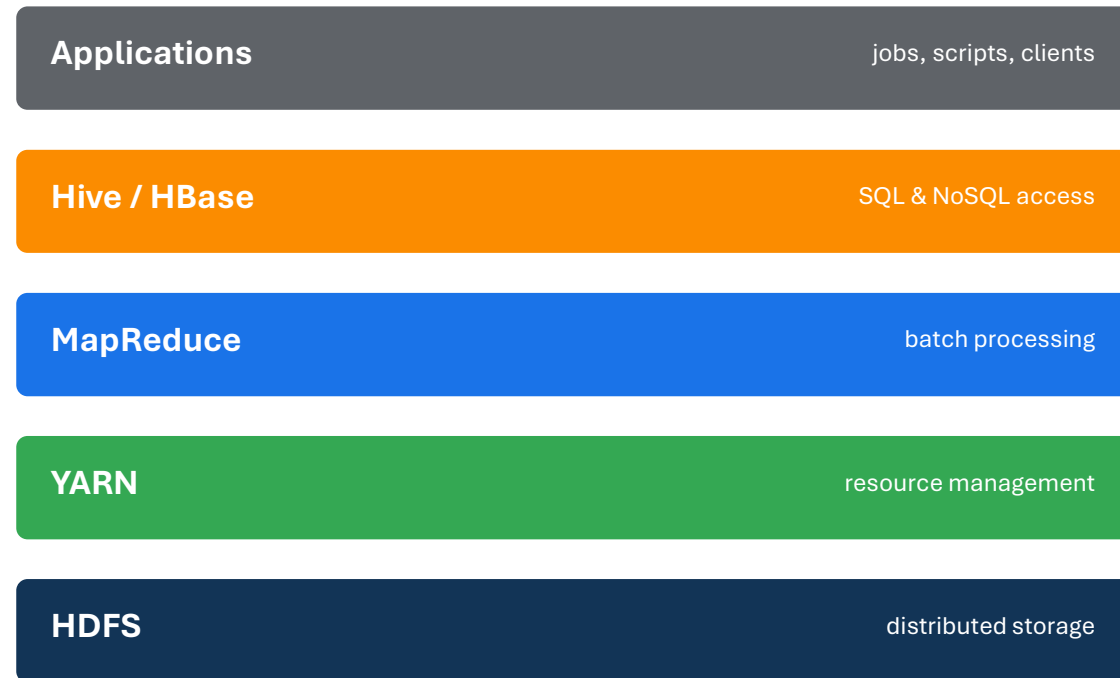
“Move compute to data, not data to compute.”

PART V

Hadoop comme plateforme

Hadoop is not one software

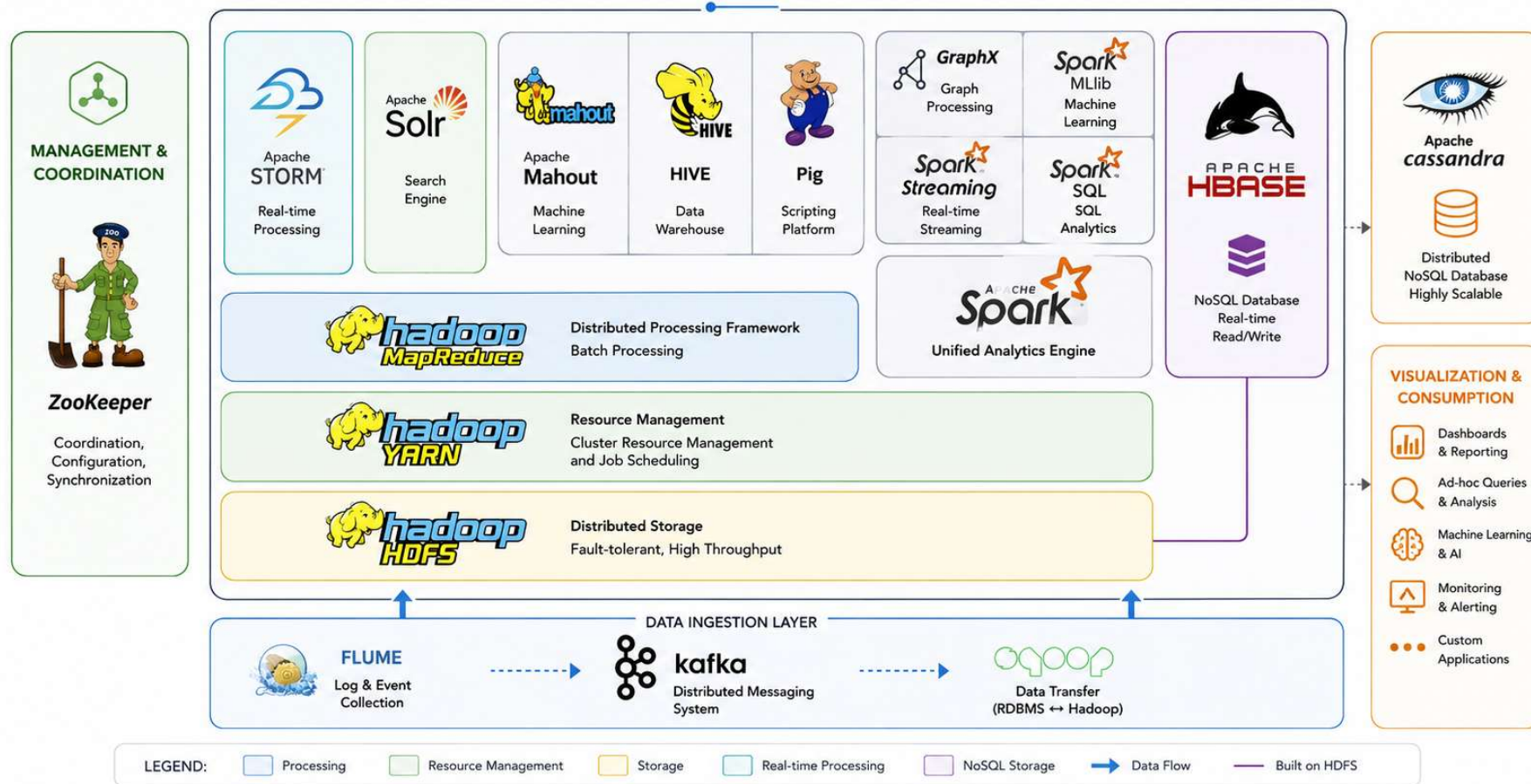
C'est un ensemble de couches qui coopèrent.



Hadoop is not one software

HADOOP ECOSYSTEM

Open Source Framework for Distributed Storage and Processing



Cinq composants, cinq responsabilités

Chaque technologie répond à un besoin précis.

HDFS

Stocker

Stockage distribué
et réplication

YARN

Orchestrer

CPU/RAM/containers
/jobs

MapReduce

Calculer

Traitement batch
distribué

Hive

Interroger

SQL sur données
massives

HBase

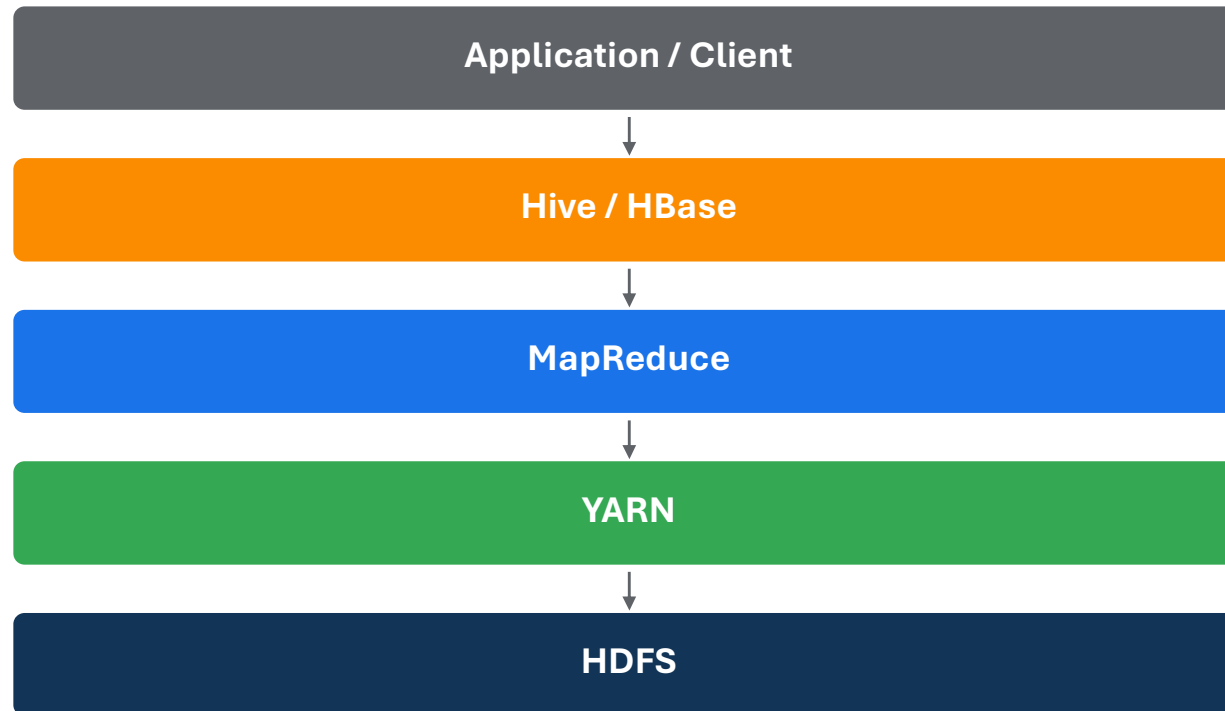
Accéder

NoSQL orienté
colonnes

Dans ce module, vous apprendrez à les assembler.

Architecture logique Hadoop

De l'application jusqu'aux blocs de données.



Chaque couche masque une partie de la complexité distribuée.

PART VI

Retour au cas métier

NovaBank : stockage à froid pour DORA

Le contexte métier du projet fil rouge.

DORA Règlement (UE) 2022/2554

Entré en application : 17 janvier 2025

Les journaux techniques doivent être conservés, exploitables et disponibles pour audit ou investigation.

Objectif pédagogique : concevoir la couche Cold Storage Hadoop.



Hot Data vs Cold Data

Toutes les données n'ont pas besoin du même stockage.

Hot Data

Données récentes

Monitoring
Recherche rapide
Investigation
Dashboards

Exemple : Elasticsearch

Cold Data

Données historiques

Archivage longue durée
Coût optimisé
Analyse ponctuelle
Audit

Exemple : Hadoop / HDFS

Le projet se concentre sur la partie Cold Storage.

Ce que vous allez construire pendant le semestre

Une plateforme Hadoop de stockage à froid et d'analyse historique.



À la fin : démonstration + architecture + justification technique.

PART VII

Environnements de travail

Trois environnements complémentaires

Apprendre, visualiser, puis travailler comme en entreprise.

Local

Docker ou Hadoop Standalone

Objectif : tester librement et développer les premiers jobs

Cloudera Demo

Vue rapide d'un écosystème Hadoop complet

Objectif : visualiser les services

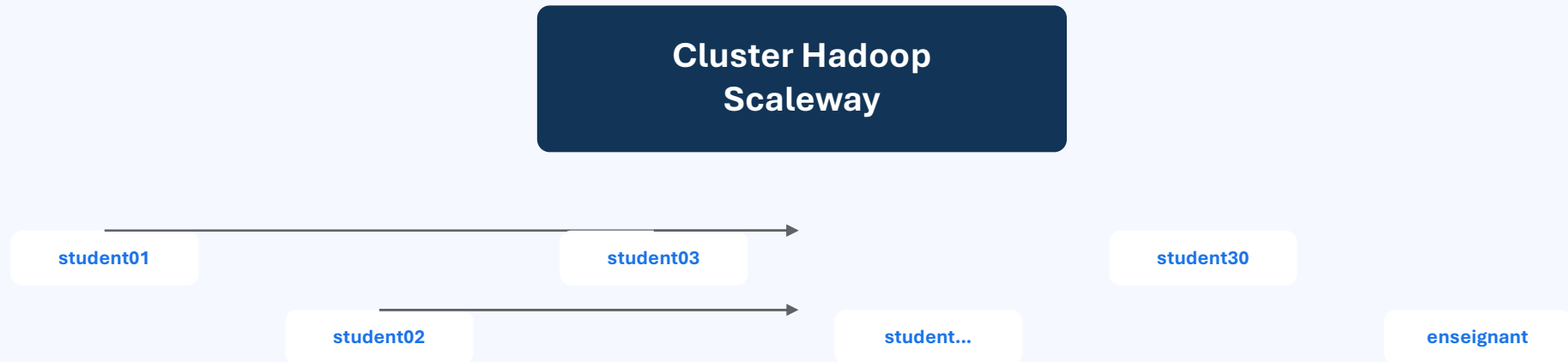
Scaleway Cluster

Plateforme mutualisée

Comptes étudiants, quotas HDFS, YARN partagé

Pourquoi un cluster mutualisé ?

C'est comme cela que fonctionnent les plateformes Big Data en entreprise.



Comptes individuels • Quotas HDFS • Files YARN • Ressources partagées

TP aujourd'hui : mise en route

Objectif : être opérationnel pour les prochaines séances.

- 1 Installer / vérifier Java
- 2 Lancer Docker ou Standalone
- 3 Découvrir Cloudera
- 4 Se connecter au cluster
- 5 Premières commandes HDFS

Key takeaways

- 1 Big Data commence quand les technologies classiques ne suffisent plus.
- 2 Hadoop répond à un problème d'architecture distribuée.
- 3 Le projet du semestre : construire la couche Cold Storage de NovaBank.

Questions ? Puis TP.